SDE1 INTERVIEW PREPARATION • 50+ Q&A

Interview Questions

& Model Answers

URL Shortener — System Design

Conceptual | Design | Coding | Trade-offs | Edge Cases

50+	7	Full
Questions	Categories	Answers

Prepared for Kunal • M.Sc. Chemistry (Organic) — University of Delhi • 2025

CONCEPTUAL QUESTIONS

Q1 What is a URL shortener and why do we need it?

Conceptual

ANSWER

A URL shortener converts a long URL into a compact short URL using a redirect mapping. We need it because long URLs are hard to share (especially on SMS and social media with character limits), look untrustworthy, break in emails, produce dense QR codes, and make campaign analytics nearly impossible. Short URLs act as a transparent redirect layer that also enables click tracking, geo-analytics, and A/B testing.

Q2 What does URL stand for and what are its components?

Conceptual

ANSWER

URL stands for Uniform Resource Locator. Its components are: Protocol (https://), Domain (www.example.com), Path (/products/item), Query Parameters (?ref=home&id=123), and Fragment (#section). Long URLs typically have many query parameters appended for tracking which makes them very long and unwieldy to share.

Q3 Why is a URL shortener considered a read-heavy system?

Conceptual

ANSWER

For every URL that is shortened (a write), it may be clicked thousands or millions of times (reads/redirects). The typical read-to-write ratio is about 10:1 or higher. At 100 million URLs shortened per day, we'd expect 1 billion redirect requests per day. This means the system must be optimized primarily for fast read/lookup operations, which is why caching with Redis is critical.

Q4 What is the difference between HTTP 301 and HTTP 302 redirects?

Conceptual

ANSWER

301 is a Permanent redirect — the browser caches it locally, so future visits to the short URL skip the server entirely. This saves server load but means you lose click analytics after the first visit. 302 is a Temporary redirect — the browser always contacts the server on every click, so you can track every visit. URL shorteners use 302 because analytics (click counts, geo-tracking, device type) are a core feature of the service.

Q5 What happens if a URL shortener uses 301 instead of 302?

Conceptual

ANSWER

With 301, the browser permanently caches the redirect. On subsequent visits to the short URL, the browser goes directly to the long URL without hitting the shortener server at all. This means: (1) You lose all click analytics after the first visit. (2) If the long URL ever changes, users with cached redirects will never see the update. (3) You cannot revoke or expire the short URL from the user's browser. These are serious limitations for a production shortener.

Q6

Can the same long URL be shortened into multiple different short URLs?

Conceptual

ANSWER

Yes, depending on implementation. If you always check for existing mappings before creating a new one (deduplication), the same long URL always returns the same short code — saving storage but preventing per-user tracking. If you allow multiple short codes per long URL, different campaigns or users can track their own clicks independently. Most production systems (like bit.ly) allow the same long URL to have multiple short codes for different tracking purposes.

Q7

Why does a URL shortener need analytics and how is it implemented?

Conceptual

ANSWER

Analytics is a primary value proposition of URL shorteners — marketers need to know how many clicks a link got, from which country, device, browser, and at what time. Implementation: on each redirect request, the server asynchronously publishes a click event to a message queue (like Kafka). A separate analytics consumer service reads from the queue and writes to an analytics database (like ClickHouse or BigQuery). This is done asynchronously so it doesn't slow down the redirect response.

DESIGN QUESTIONS

Q8 How would you design the database schema for a URL shortener? Design

■ ANSWER

Main table url_mapping with columns: id (BIGINT, auto-increment primary key), short_code (VARCHAR(10), unique index), long_url (TEXT, not null), user_id (BIGINT, foreign key to users table), created_at (DATETIME, default NOW()), expires_at (DATETIME, nullable for optional expiry), click_count (INT, default 0). Create a unique index on short_code for O(1) lookup. Optionally maintain a separate url_clicks table for detailed per-click analytics (ip, country, device, timestamp).

Q9 SQL or NoSQL for a URL shortener — which would you choose and why? Design

■ ANSWER

For the core short_code → long_url mapping, NoSQL (like Cassandra or DynamoDB) is preferred at large scale. The primary access pattern is a single key lookup (short_code), which NoSQL key-value stores handle extremely efficiently with no joins. Cassandra offers linear horizontal scalability and high availability. However, SQL (MySQL/PostgreSQL) works perfectly for small-to-medium scale and offers better query flexibility. In interviews, either answer is valid if you justify the trade-offs.

Q10 How does caching help in a URL shortener? What eviction policy would you use? Design

■ ANSWER

Since the system is read-heavy (10:1 ratio), caching the short_code → long_url mapping in Redis dramatically reduces DB load. On a redirect request, the server first checks Redis — a cache hit returns the long URL in microseconds without touching the DB. Only on a cache miss does the server query the DB, then populate Redis for future requests. Use LRU (Least Recently Used) eviction policy so popular short URLs stay warm in cache while rarely accessed ones are evicted. Set a reasonable TTL (e.g., 24 hours) to handle URL updates.

Q11 How would you design the system to handle URL expiry? Design

■ ANSWER

Add an expires_at column (nullable DATETIME) to the url_mapping table. On every redirect request, after fetching the long URL, check if expires_at is set and in the past — if so, return HTTP 410 Gone (not 404, because the resource existed but is now expired). Run a background cron job (daily or hourly) to DELETE expired rows from the DB and invalidate their Redis cache entries using DEL short_code. Expired short codes can be added back to the available pool for reuse.

Q12 **What components make up a complete URL shortener architecture?** Design

■ ANSWER

A production URL shortener consists of: (1) Load Balancer — distributes traffic across app servers. (2) Application Servers — handle shorten/redirect logic. (3) ZooKeeper / ID Generator — produces unique IDs without collision. (4) Redis Cache — stores hot short_code mappings for fast reads. (5) Primary Database — persists all short_code → long_url mappings. (6) Read Replicas — distribute DB read load. (7) Kafka Queue — receives click events asynchronously. (8) Analytics Service — consumes Kafka events for reporting.

Q13 **How would you handle custom aliases (vanity URLs) in the system?** Design

■ ANSWER

When a user requests a custom alias (e.g., bit.ly/my-brand): (1) Check the url_mapping table — if short_code = 'my-brand' exists, return 409 Conflict. (2) Maintain a reserved_codes table or blacklist for system words like 'api', 'admin', 'health', 'stats' that can never be aliases. (3) Validate alias format (alphanumeric only, max 20 chars). (4) If available, insert directly into url_mapping with the custom alias as short_code, bypassing the normal ID generation flow.

Q14 **How would you make the redirect API as fast as possible?** Design

■ ANSWER

Multiple layers of optimization: (1) Use Redis cache as first lookup — microsecond response time. (2) Keep app server logic minimal — just a cache/DB lookup and HTTP redirect, no heavy processing. (3) Use HTTP keep-alive connections to the cache and DB. (4) Log analytics asynchronously via Kafka — never block the redirect for logging. (5) Deploy servers geographically close to users (CDN/edge). (6) Use connection pooling for DB connections. (7) Return HTTP 302 which is lightweight. Target: < 10ms p99 redirect latency.

CODING QUESTIONS

Q15 **Implement Base62 encoding (integer to short code).** Coding

■ ANSWER

alphabet = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ' def encode(num): result = []; while num > 0: result.append(alphabet[num % 62]); num //= 62 return ''.join(reversed(result)) or '0' Example: encode(125000) → 'ww8'. Algorithm: repeatedly divide by 62, collect remainders as alphabet indices, reverse the collected characters.

Q16 **Implement Base62 decoding (short code to integer).** Coding

■ ANSWER

alphabet = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ' char_map = {c: i for i, c in enumerate(alphabet)} def decode(code): result = 0 for ch in code: result = result * 62 + char_map[ch] return result Example: decode('ww8') → 32×62² + 32×62¹ + 8×62■ = 123008 + 1984 + 8 = 125000.

Q17 **Implement an in-memory URL shortener class with shorten and expand methods.** Coding

■ ANSWER

class URLShortener: def __init__(self): self.db = {}; self.counter = 1 def shorten(self, long_url): code = self._encode(self.counter); self.counter += 1 self.db[code] = long_url; return 'https://short.ly/' + code def expand(self, short_url): code = short_url.split('/')[-1] return self.db.get(code, 'Not found') def _encode(self, n): alpha = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ' res = [] while n > 0: res.append(alpha[n%62]); n //= 62 return ''.join(reversed(res))

Q18 **How would you make the URL shortener thread-safe in a multi-threaded environment?** Coding

■ ANSWER

The main race condition is on ID generation. Solutions: (1) Use a thread-safe atomic counter — in Python, use threading.Lock() around counter increment. (2) Use Redis INCR command — Redis is single-threaded internally, so INCR is atomic by design; multiple app servers can safely call INCR concurrently. (3) Use DB AUTO_INCREMENT with INSERT — the DB handles atomicity and returns the generated ID. (4) Use Snowflake IDs — the machine_id component ensures uniqueness across servers without any shared state.

Q19 **How would you add expiry support to the in-memory URL shortener?** Coding

■ ANSWER

Extend the storage to save expiry time alongside the URL: self.db[code] = {'url': long_url, 'expires_at': time.time() + ttl_seconds}. In the expand method, before returning, check: if record['expires_at'] and time.time() > record['expires_at']: del self.db[code]; return 'Link expired'. Run a background thread with a cleanup loop: periodically iterate self.db and delete all entries where expires_at < time.time(). This mirrors how Redis TTL and a cron job work in production.

Q20

Why is Base62 preferred over Base64 for URL shortening?

Coding

■ ANSWER

Base64 uses 64 characters: A-Z, a-z, 0-9, plus '+' and '/'. The '+' and '/' characters are special in URLs and must be percent-encoded as %2B and %2F, which makes the short URL longer and less clean. Base62 uses only alphanumeric characters (A-Z, a-z, 0-9) — all of which are URL-safe with no encoding needed. The trade-off is minimal: Base64 gives 64■ ≈ 4.4 trillion combinations vs Base62's 62■ ≈ 3.5 trillion — the loss is negligible.

Q21

How many unique short URLs can you generate with 7 Base62 characters?

Coding

■ ANSWER

62■ = 62 × 62 × 62 × 62 × 62 × 62 × 62 = 3,521,614,606,208 ≈ 3.5 trillion unique codes. At 100 million new URLs per day, this would take 3.5 trillion ÷ 100 million = 35,000 days ≈ 95 years before exhaustion. In practice, 7 characters is sufficient for any foreseeable production system. If needed, simply increase to 8 characters for 218 trillion combinations.

TRADE-OFFS QUESTIONS

Q22 **Why is hashing (MD5/SHA) a bad approach for generating short codes?** Trade-offs

ANSWER

MD5/SHA produce fixed-length hashes, but when you truncate to 6-7 characters to get a short code, different URLs can produce the same prefix — this is a hash collision. Resolving collisions requires a DB lookup for every generated code ('is this taken?') and a retry with modified input if it is. This creates a slow, complex retry loop that worsens under high load. Additionally, hashes give no ordering or operational visibility. Base62 encoding of a unique ID has zero collisions by construction since each ID is globally unique.

Q23 **Compare auto-increment ID vs Snowflake ID for unique ID generation.** Trade-offs

ANSWER

Auto-increment: Simple, guaranteed unique, ordered — but creates a single point of failure (one DB), becomes a write bottleneck at scale, and produces sequential IDs that are easily enumerable by attackers (bit.ly/000001, /000002, /000003...). Snowflake ID: Distributed (each server uses its own machine_id), generates 4096 IDs/ms per server, time-sortable, no central coordinator needed — but requires clock synchronization (NTP) and clock skew can cause issues. For large-scale production, Snowflake is preferred.

Q24 **What are the trade-offs between short code length (5 vs 7 vs 8 chars)?** Trade-offs

ANSWER

Shorter codes are cleaner and more memorable but support fewer unique URLs. 5 chars = 916M combinations (exhausted quickly at scale). 6 chars = 56B combinations (adequate for medium platforms). 7 chars = 3.5T combinations (recommended, lasts 95+ years at 100M/day). 8 chars = 218T combinations (future-proof but slightly longer). Most production systems use 6-7 characters as the sweet spot between brevity and capacity.

Q25 **Pre-generated ID pool vs on-demand ID generation — pros and cons?** Trade-offs

ANSWER

Pre-generated pool: Generate IDs in bulk and store in Redis. Servers pop IDs instantly with no computation. Pros: extremely fast, no coordination per ID. Cons: complex to implement and maintain, IDs wasted if server crashes before using its batch, pool refill logic adds operational overhead. On-demand generation (Snowflake): Generate each ID when needed using timestamp + machine_id + sequence. Pros: simpler, no pool management, naturally distributed. Cons: slightly more computation per request, requires clock sync. For most systems, Snowflake is simpler and sufficiently fast.

Q26

When would you choose Redis over a database for storing short URL mappings?

Trade-offs

ANSWER

Redis alone is not suitable for primary storage because it is in-memory and data is lost on restart (unless persistence is configured). Use Redis as a cache layer on top of a persistent DB. The pattern: write to DB always (persistent, queryable), cache in Redis on read (fast, temporary). Redis is ideal because short_code lookups are a pure key-value operation — exactly what Redis is optimized for. Use LRU eviction to keep hot codes in memory and evict cold ones. Only fall back to DB on cache miss.

EDGE CASES QUESTIONS

Q27 What happens if two users request the same custom alias simultaneously?

Edge Cases

ANSWER

This is a race condition on custom alias insertion. Solution: Use a DB UNIQUE constraint on short_code. Both servers attempt INSERT INTO url_mapping (short_code=...) simultaneously. The DB serializes the inserts — one succeeds and one fails with a UNIQUE constraint violation. The server catching the constraint error returns HTTP 409 Conflict to that user with the message 'Alias already taken.' The DB constraint is the final arbiter of uniqueness. You can also use SELECT FOR UPDATE to lock the row before attempting insert.

Q28 How do you handle a request to shorten a URL that is itself a short URL?

Edge Cases

ANSWER

Shortening a short URL creates a redirect chain (bit.ly/abc → bit.ly/xyz → final URL), which adds unnecessary latency. Solution: Before creating a new mapping, resolve the input URL by following its redirects. If it resolves to a URL that is already in our system (or is another shortener's URL), store the final resolved long URL instead of the intermediate short URL. Also set a maximum redirect chain depth (e.g., 3) on the redirect endpoint to prevent infinite loops from circular redirects.

Q29 What do you return when a user visits an expired short URL?

Edge Cases

ANSWER

Return HTTP 410 Gone (not 404 Not Found). The distinction matters: 404 means the resource never existed or the URL is wrong. 410 means the resource existed but has been permanently removed — which is exactly what an expired short URL represents. The response body should include a user-friendly message like 'This link has expired.' Also ensure the expired code is removed from Redis cache so future requests don't serve stale data from cache.

Q30 How do you handle malicious or invalid URLs submitted for shortening?

Edge Cases

ANSWER

Multi-layer validation: (1) Format check — validate the URL has a valid schema (https://) and domain using a URL parser, reject malformed inputs with 400 Bad Request. (2) Blacklist check — maintain a list of known malicious domains, reject matches with 403 Forbidden. (3) Safe Browsing API — call Google Safe Browsing API (or similar) to check the URL against known phishing/malware databases. (4) Rate limiting — limit shortening requests per user/IP to prevent abuse. (5) NSFW detection — optionally scan page content before shortening.

Q31

What is a race condition in URL shortener and how do you prevent it?

Edge Cases

■ ANSWER

A race condition occurs when two servers simultaneously generate the same ID and both try to insert a record with the same short_code into the DB. Prevention strategies: (1) DB UNIQUE constraint — insert will fail for the duplicate, catch the exception and retry. (2) Atomic Redis INCR — Redis processes INCR atomically, ensuring each server gets a different counter value. (3) Snowflake IDs — the machine_id component ensures two servers never produce the same ID at the same millisecond. (4) ZooKeeper token ranges — assign non-overlapping ID ranges to each server so ranges never intersect.

Q32

How do you prevent users from enumerating all short URLs in the system?

Edge Cases

■ ANSWER

Sequential short codes (000001, 000002...) let attackers scrape the entire URL database by iterating through codes. Prevention: (1) Base62-encode large, non-sequential IDs — even sequential IDs like 1,000,000 and 1,000,001 produce codes like '4c92' and '4c93' that appear random. (2) Use Snowflake IDs which embed a timestamp and machine component, making output look random. (3) Add a random salt to the ID before encoding. (4) Rate-limit the redirect endpoint per IP — excessive 404s trigger a temporary block. (5) Require authentication for sensitive private links.

Q33

What happens if the Redis cache goes down?

Edge Cases

■ ANSWER

If Redis is unavailable, all redirect requests fall through to the database. The system remains functional but slower — the DB must handle the full read load (~11,500 reads/sec in our estimates). To handle this gracefully: (1) Implement circuit breaker pattern — if Redis fails, immediately route to DB without waiting for Redis timeout. (2) Auto-reconnect logic — the app server should retry Redis connection in the background. (3) DB read replicas absorb the sudden load increase. (4) Alert on-call engineers immediately. Redis downtime is recoverable; the cache simply rebuilds itself from DB as requests come in.

ESTIMATION QUESTIONS

Q34 **Estimate writes per second for a system shortening 100 million URLs per day.** Estimation

ANSWER

Writes/sec = 100,000,000 ÷ 86,400 seconds/day ≈ 1,157 writes/sec. This is a moderate write load that a single well-tuned MySQL instance can handle (MySQL handles ~5,000 simple writes/sec). However, for safety margin and high availability, you'd use a primary DB with automated failover. At peak hours, you might see 2-3x the average, so plan for ~3,500 writes/sec burst capacity.

Q35 **Estimate reads per second for a 10:1 read-to-write ratio at 100M URLs/day.** Estimation

ANSWER

If 100M URLs are shortened per day (writes), with a 10:1 read-to-write ratio, we get 1 billion redirects per day. Reads/sec = 1,000,000,000 ÷ 86,400 ≈ 11,574 reads/sec average. This is too high for a single DB without caching. With Redis cache and ~80% hit rate, only 2,315 read/sec actually hit the DB — easily handled by a primary + 2 read replicas.

Q36 **Estimate storage requirements for 5 years of URL shortener data.** Estimation

ANSWER

Per URL record: short_code (10 bytes) + long_url (avg 200 bytes) + metadata (100 bytes) ≈ 310 bytes ≈ 500 bytes with overhead. Daily: 100M × 500 bytes = 50 GB/day. Yearly: 50 GB × 365 = 18.25 TB/year. 5-year total: 18.25 × 5 ≈ 91 TB. This fits comfortably on modern cloud storage (S3, GCS) at low cost. The DB itself would hold active/recent URLs; older data can be archived to cold storage.

Q37 **How many servers would you need to handle 11,500 reads per second?** Estimation

ANSWER

Assuming each app server can handle ~2,000 redirect requests/sec (with Redis cache, the bottleneck is network I/O not CPU), you need 11,500 ÷ 2,000 ≈ 6 app servers. Add 50% buffer for failures and traffic spikes → 9 app servers. In practice, with auto-scaling on cloud, you'd set minimum 3 servers and scale to 10+ during peak. The load balancer distributes traffic using round-robin or least-connections algorithm.

API DESIGN QUESTIONS

Q38 **Design the complete REST API for a URL shortener.** API Design

■ ANSWER

POST /api/shorten — Body: {long_url, custom_alias?, ttl_seconds?} → 201 {short_url, expires_at} | 400 bad URL | 409 alias taken. GET /{short_code} → 302 redirect to long_url | 404 not found | 410 expired. GET /api/stats/{short_code} → 200 {clicks, top_countries, devices, created_at} | 403 unauthorized | 404 not found. DELETE /api/{short_code} → 200 OK | 403 forbidden (not owner) | 404 not found. PUT /api/{short_code} — Body: {long_url} → 200 updated | 403 forbidden.

Q39 **What HTTP status codes are most important for a URL shortener API?** API Design

■ ANSWER

201 Created — new short URL successfully created. 302 Found — redirect response (used instead of 301 to preserve analytics). 400 Bad Request — invalid URL format submitted. 403 Forbidden — user doesn't have permission (e.g., trying to delete another user's URL). 404 Not Found — short code doesn't exist in the system. 409 Conflict — custom alias already taken by another URL. 410 Gone — short URL existed but has expired. 429 Too Many Requests — rate limit exceeded. 500 Internal Server Error — unexpected server failure.

Q40 **How would you implement rate limiting on the URL shortener API?** API Design

■ ANSWER

Use a sliding window rate limiter implemented in Redis. For each user/IP, store a sorted set in Redis with request timestamps as scores. On each request: (1) Remove timestamps older than the window (e.g., 1 hour). (2) Count remaining entries. (3) If count >= limit (e.g., 1000/hour), return 429 Too Many Requests with Retry-After header. (4) Else, add current timestamp and proceed. Apply stricter limits to the POST /shorten endpoint (write-heavy abuse risk) and looser limits to GET /{code} (redirect is read-only and less abusable). Use different limits for authenticated vs anonymous users.

Q41 **How would you version the URL shortener API?** API Design

■ ANSWER

Use URL path versioning as the primary strategy: /api/v1/shorten, /api/v2/shorten. This is explicit, easy to route in load balancers, and clear in documentation. Avoid header-based versioning (X-API-Version) as it's invisible in browser URLs and harder to cache. Maintain v1 for at least 12 months after v2 launch with deprecation warnings in response headers (Deprecation: true, Sunset: date). Use semantic versioning internally: v1 is stable API, v2 introduces breaking changes (new required fields, changed response structure).